

Material编译过程及目标文件格式

编译过程

在filament静态库编译过程中，有一系列扩展名为.mat的文件需要编译成.filamat，并将他们组合在一起，并最终和其它源文件一同编译为libfilament.a。

编译材质定义文件

首先，我们看原始的材质定义文件：

```

liwei66@xiaomi:~/work/L3-system/external/MREngine$ tree filament/src/materials/
filament/src/materials/
├── antiAliasing
│   ├── fxaa.fs
│   ├── fxaa.mat
│   └── taa.mat
├── blitLow.mat
├── bloom
│   ├── bloomDownsample.mat
│   └── bloomUpsample.mat
├── colorGrading
│   ├── colorGradingAsSubpass.mat
│   ├── colorGrading.fs
│   ├── colorGrading.mat
│   └── customResolveAsSubpass.mat
├── debugShadowCascades.mat
├── defaultMaterial0.mat
├── defaultMaterial.mat
├── dof
│   ├── dofCoc.mat
│   ├── dofCombine.mat
│   ├── dofDilate.mat
│   ├── dofDownsample.mat
│   ├── dof.mat
│   ├── dofMedian.mat
│   ├── dofMipmap.mat
│   ├── dofTiles.mat
│   ├── dofTilesSwizzle.mat
│   └── dofUtils.fs
├── flare
│   └── flare.mat
├── fsr
│   ├── ffx_a.h
│   ├── ffx_fsr1.h
│   ├── ffx_fsr1_mobile.fs
│   ├── fsr_easu.mat
│   ├── fsr_easu_mobileF.mat
│   ├── fsr_easu_mobile.mat
│   ├── fsr_rcas.mat
│   └── license.txt
├── separableGaussianBlur.fs
├── separableGaussianBlur.mat
├── separableGaussianBlur.vs
├── skybox0.mat
├── skybox.mat
├── ssao
│   ├── bilateralBlurBentNormals.mat
│   ├── bilateralBlur.mat
│   ├── depthUtils.fs
│   ├── geometry.fs
│   ├── mipmapDepth.mat
│   ├── saoBentNormals.mat
│   ├── saoImpl.fs
│   ├── sao.mat
│   ├── ssaoUtils.fs
│   ├── ssct.fs
│   └── ssctImpl.fs
└── vsmMipmap.mat

7 directories, 49 files

```

其中以.mat结尾的文件都是，而.fs和.vs是片段着色和定点着色器脚本，在编译材质文件的过程中，会包含到材质代码中一同编译。

编译.mat文件，需要使用工具matc，matc的说明如下：

```
matc is a command-line tool to compile material definition.

Caution! matc was designed to operate on trusted inputs. To minimize the risk of
triggering memory corruption vulnerabilities, please make sure that the files passed
to matc come from a trusted source, or run matc in a sandboxed environment.

Usages:
  matc [options] <input-file>

Supported input formats:
  Filament material definition (.mat)
```

通过文档说明，看到matc是用来编译材质定义的命令行工具。但我们需要保证输入的文件是可信的，或者在沙箱环境中运行matc。而我们Android.bp中，调用外部命令执行特殊操作的过程就是在沙箱中运行的。但想要编译出最终文件，还是需要确保我们的文件是可用的。

我们编译脚本配置如下：

```
gensrcs {
    name: "materials_fsr_easu",

    tools: ["matc"],
    cmd: "$(location matc) -p mobile -a opengl -a vulkan -o $(out) $(in)",
    srcs: [
        "src/materials/fsr/fsr_easu.mat",
    ],
    output_extension: "mat",
}
```

其中核心的指令为：

```
"$(location matc) -p mobile -a opengl -a vulkan -o $(out) $(in)"
```

其中，主要用到了-p、-a、-o三个参数，matc参数说明如下：

```
Options:
  --help, -h
    Print this message

  --license
    Print copyright and license information

  --output, -o
    Specify path to output file

  --platform, -p
    Shader family to generate: desktop, mobile or all (default)

  --optimize-size, -S
    Optimize generated shader code for size instead of just performance

  --api, -a
    Specify the target API: opengl (default), vulkan, metal, or all
    This flag can be repeated to individually select APIs for inclusion:
    matc --api opengl --api metal ...

  --define, -D
    Add a preprocessor define macro via <macro>=<value>. <value> defaults to 1 if omitted.
    Can be repeated to specify multiple definitions:
    matc -Dfoo=1 -Dbar -Dbuzz=100 ...

  --template <macro>=<string>, -T<macro>=<string>
    Replaces ${MACRO} with specified string before parsing
    Unlike --define, this applies to the material specification, not GLSL.
    Can be repeated to specify multiple macros:
    matc -TBLENDING=fade -TDOUBLESIDED=false ...

  --reflect, -r
    Reflect the specified metadata as JSON: parameters

  --variant-filter=<filter>, -V <filter>
    Filter out specified comma-separated variants:
    directionalLighting, dynamicLighting, shadowReceiver, skinning, vsm, fog, ssr (screen-space reflections)
    This variant filter is merged with the filter from the material, if any

  --version, -v
    Print the material version number

Internal use and debugging only:
  --optimize-none, -g
    Disable all shader optimizations, for debugging

  --preprocessor-only, -E
    Optimize shaders by running only the preprocessor

  --raw, -w
    Compile a raw GLSL shader into a SPIRV binary chunk

  --output-format, -f
    Specify output format: blob (default) or header

  --debug, -d
    Generate extra data for debugging

  --print, -t
    Print generated shaders for debugging
```

可以看到：

`-p` 指定的是平台，用来指定着色器的家族，这里配置为`mobile`，为移动平台。

`-a` 指定的是目标API，这里配置了两个目标，一个是`vulkan`，一个是`opengl`。

`-o` 指定的是结果输出路径，由于是在沙箱中运行，我们这里将输入和输出扩展名统一为`.mat`，并没有使用`.filamat`，但不影响。

组合材质目标文件

从上面的编译过程说明，我们知道，每个`.mat`文件都生成了一个同名的`.mat`目标文件，而下一步就是将这些文件组合成一个，生成对应的头文件和源文件。

我们这个过程使用的是`resgen`工具。`resgen`说明如下：

resgen aggregates a sequence of binary blobs, each of which becomes a "resource" whose id is the basename of the input file. It produces the following set of files:

```
resources.h ..... declares sizes and offsets for each resource
resources.S ..... small assembly file with incbin directive and rodata section
resources.apple.S ... ditto but with different rodata name and underscore prefixes
resources.bin ..... the aggregated binary blob that the incbin refers to
```

Usage:

```
resgen [options] <input_file_0> <input_file_1> ...
```

resgen用来组合一系列二进制Blob，它们会变成一个和输入文件名称一样扩展名不同的资源，这个资源包括了上面四个文件。

对应的编译脚本指令为：

```
cmd: "export MAT_SRCS=\"$(in)\"; $(location gen_materials) $(location resgen) $(out) $(genDir)",
```

而gen_materials中核心实现为：

```
$1 -qcx $3 -p materials ${MAT_SRCS}
```

相当于：

```
resgen -qcx output_dir -p materials xxx.mat yyy.mat
```

我们用到的是q、c、x、p四个参数。

其中的参数说明：

Options:

```
--help, -h
    Print this message
--license, -L
    Print copyright and license information
--package=string, -p string
    Name of the resource package (defaults to "resources")
    This is used to generate filenames and symbol prefixes
--deploy=dir, -x dir (defaults to ".")
    Generate everything needed for deployment into <dir>
--keep, -k
    Keep file extensions when generating symbols
--text, -t
    Append a null terminator to each data blob
--cfile, -c
    Generate xxd-style C file (useful for WebAssembly)
--json, -j
    Embed a JSON string in the output that provides a summary
    of all resource sizes and names. Useful for size analysis.
--quiet, -q
    Suppress console output
```

Examples:

```
resgen -cp textures jungle.png beach.png
> Generated files: textures.h, textures.S, textures.apple.S, textures.bin, textures.c
> Generated symbols: TEXTURES_JUNGLE_DATA, TEXTURES_JUNGLE_SIZE,
    TEXTURES_BEACH_DATA, TEXTURES_BEACH_SIZE
```

可以看到：

-q 控制台不输出日志

-c 生成xxd类型的C文件（配置了之后，就不会再生成.S文件，我们生成.c文件，可读性更好）

-x 指定输出目录

-p 指定最终资源包的名称，这里指定的是materials，也就是生成的目标为materials（头文件为materials.h，源文件为materials.c等）

将材质资源源文件编译为最终库

这里实际上就是filament静态库的编译过程，也就是将上面生成的材质源文件加入到静态库的源文件列表中一并编译的过程，这里不多介绍。

文件格式

这里说的文件格式，既是最终静态库中资源的分布方式，也是参与编译的资源源文件的格式。

materials.h

```
#ifndef MATERIALS_H_
#define MATERIALS_H_

#include <stdint.h>

extern "C" {
    extern const uint8_t MATERIALS_PACKAGE[];
    extern int MATERIALS_FSR_RCAS_OFFSET;
    extern int MATERIALS_FSR_RCAS_SIZE;
    extern int MATERIALS_FSR_EASU_OFFSET;
    extern int MATERIALS_FSR_EASU_SIZE;
    extern int MATERIALS_FSR_EASU_MOBILE_OFFSET;
    extern int MATERIALS_FSR_EASU_MOBILE_SIZE;
    extern int MATERIALS_FSR_EASU_MOBILEF_OFFSET;
    extern int MATERIALS_FSR_EASU_MOBILEF_SIZE;
    ...
    extern int MATERIALS_DEFAULTMATERIAL_OFFSET;
    extern int MATERIALS_DEFAULTMATERIAL_SIZE;
    ...
}
#define MATERIALS_FSR_RCAS_DATA (MATERIALS_PACKAGE + MATERIALS_FSR_RCAS_OFFSET)
#define MATERIALS_FSR_EASU_DATA (MATERIALS_PACKAGE + MATERIALS_FSR_EASU_OFFSET)
#define MATERIALS_FSR_EASU_MOBILE_DATA (MATERIALS_PACKAGE + MATERIALS_FSR_EASU_MOBILE_OFFSET)
#define MATERIALS_FSR_EASU_MOBILEF_DATA (MATERIALS_PACKAGE + MATERIALS_FSR_EASU_MOBILEF_OFFSET)
...
#define MATERIALS_DEFAULTMATERIAL_DATA (MATERIALS_PACKAGE + MATERIALS_DEFAULTMATERIAL_OFFSET)
...

#endif
```

可以看到，这里面主要定义了：

1. 名为MATERIALS_PACKAGE的uint8_t类型数组
2. 一系列_OFFSET和_SIZE的int值外部变量，变量实现在materials.c中
3. 一系列_DATA的宏

以DEFAULTMATERIAL为例，我们通过对宏MATERIALS_DEFAULTMATERIAL_DATA定义的分析，可以知道，就是定义了DEFAULTMATERIAL在MATERIALS_PACKAGE数组中的位置，其位置是通过1中的数组起始地址（MATERIALS_PACKAGE）和2中定义的偏移量（MATERIALS_DEFAULTMATERIAL_OFFSET）来计算的。

materials.c

```

#include <stdint.h>

const uint8_t MATERIALS_PACKAGE[] = {
// FSR_EASU
0x53, 0x52, 0x45, 0x56, 0x5f, 0x54, 0x41, 0x4d, 0x04, 0x00, 0x00, 0x00, 0x25, 0x00, 0x00, 0x00, 0x54, 0x41,
0x45, 0x46,
...
0x00, 0x00, 0x01, 0x01, 0x01, 0x01, 0x00, 0x00, 0x00,

// FSR_EASU_MOBILE
0x53, 0x52, 0x45, 0x56, 0x5f, 0x54, 0x41, 0x4d, 0x04, 0x00, 0x00, 0x00, 0x25, 0x00, 0x00, 0x00, 0x54, 0x41,
0x45, 0x46,
...
0x00,

// FSR_EASU_MOBILEF
0x53, 0x52, 0x45, 0x56, 0x5f, 0x54, 0x41, 0x4d, 0x04, 0x00, 0x00, 0x00, 0x25, 0x00, 0x00, 0x00, 0x54, 0x41,
0x45, 0x46,
...
0x01, 0x01, 0x00, 0x00, 0x00,

// FSR_RCAS
0x53, 0x52, 0x45, 0x56, 0x5f, 0x54, 0x41, 0x4d, 0x04, 0x00, 0x00, 0x00, 0x25, 0x00, 0x00, 0x00, 0x54, 0x41,
0x45, 0x46,
...

// DEFAULTMATERIAL
0x53, 0x52, 0x45, 0x56, 0x5f, 0x54, 0x41, 0x4d, 0x04, 0x00, 0x00, 0x00, 0x25, 0x00, 0x00, 0x00, 0x54, 0x41,
0x45, 0x46,
...
};

int MATERIALS_FSR_EASU_OFFSET = 0;
int MATERIALS_FSR_EASU_SIZE = 23769;
int MATERIALS_FSR_EASU_MOBILE_OFFSET = 23769;
int MATERIALS_FSR_EASU_MOBILE_SIZE = 16861;
int MATERIALS_FSR_EASU_MOBILEF_OFFSET = 40630;
int MATERIALS_FSR_EASU_MOBILEF_SIZE = 5625;
int MATERIALS_FSR_RCAS_OFFSET = 46255;
int MATERIALS_FSR_RCAS_SIZE = 10425;
...
int MATERIALS_DEFAULTMATERIAL_OFFSET = 98014;
int MATERIALS_DEFAULTMATERIAL_SIZE = 22793;
...

```

通过对头文件的分析，再结合源文件中的数组定义和OFFSET、SIZE定义，我们可以知道：

在最终代码中，实际上只有一个大数组，我们通过源文件中的OFFSET和SIZE就能定义每个材质资源的起始地址和长度，使用过程就是使用他们来访问不同的资源的：

```

defaultMaterialBuilder.package(
    MATERIALS_DEFAULTMATERIAL_DATA, MATERIALS_DEFAULTMATERIAL_SIZE);

```

可以看到，在创建材质的过程中，我们传入的是材质数据起始地址MATERIALS_DEFAULTMATERIAL_DATA（就是从MATERIALS_PACKAGE数组）和材质的长度大小（MATERIALS_DEFAULTMATERIAL_SIZE）。

总的来说，所有这些材质资源都是通过一个大数组来保存的，而数组是连续的，这也就实现了资源的平坦排布，并且每个资源的起点和长度我们都是知道的，这样就能实现一个文件承载所有预置资源且资源访问方式统一的目的。

